Requirements Analysis and Architectural Blueprint for a Decentralized.NET MAUI Android Background Service

Acknowledgment of Requirements and Initial Assessment

The following analysis details the primary objective: the design and implementation of a highly optimized, lightweight background service that facilitates direct peer-to-peer (P2P) connections without reliance on centralized server infrastructure. The architectural paradigm requires the service to initiate automatically upon device boot within the Android operating system. Furthermore, the service must support a credential system strictly bound to the user's mobile telecommunications number, secure local data storage accessible only post-authentication, an out-of-band user discovery mechanism utilizing Short Message Service (SMS) payloads, and a robust Inter-Process Communication (IPC) layer enabling third-party applications to route arbitrary data payloads (e.g., messaging data) through the P2P daemon. The deployment strategy mandates an initial phase dedicated to development and debugging configurations, ensuring maximum visibility into network routing and cryptographic handshakes.

While the ultimate deployment target is the Android operating system, the engineering approach leverages the.NET Multi-platform App UI (.NET MAUI) framework.[1] This framework selection aligns with advanced Windows-based development methodologies, allowing the application to be written in C# while cross-compiling to native Android binaries.[1] This approach benefits from the robust networking capabilities of the.NET ecosystem, mature dependency injection frameworks, and seamless integration with native Android Java/Kotlin APIs via managed bindings.[3] The resulting architecture operates as a headless daemon, isolating complex networking logic from user-facing application clients and maximizing code reusability across potential future platforms.

To ensure the architecture precisely matches the operational environment, the development process requires iterative clarification. Several critical edge cases must be addressed before the finalization of the codebase. The overarching network topology must be defined, specifically regarding whether the P2P routing will utilize raw Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) sockets with Session Traversal Utilities for NAT (STUN) and Traversal Using Relays around NAT (TURN) fallbacks, or if it will leverage established frameworks such as the Android WifiP2pManager wrapped in C#.[4] Furthermore, the exact maximum threshold for group chat participants must be clarified, as this directly dictates the selection of the cryptographic key-exchange algorithm. The conflict resolution strategy for

instances where a single mobile number is concurrently registered on multiple physical devices also requires explicit definition, particularly in light of stringent geopolitical telecommunications regulations. Finally, the expected format of the final deliverable must be confirmed, whether it is a compiled .apk/.aab package, a unified Visual Studio .sln project, or a standalone Android Archive (.aar) library for consumption by other developers.

Version Control and Project Synchronization Protocols

A structured development lifecycle mandates absolute clarity regarding source code synchronization and configuration management. The development architecture operates under the strict principle that the provided project files, repository states, and .csproj configurations serve as the single, immutable source of truth. Internal generative memory or generalized assumptions regarding the state of the codebase are strictly prohibited to prevent integration conflicts and dependency regressions.

Before any structural edits or code generation procedures are executed, the latest iteration of the project files must be provided to the development environment. This protocol ensures that all integration points, particularly those involving Android-specific manifest configurations (AndroidManifest.xml) and.NET MAUI dependency injection registries (MauiProgram.cs), are entirely consistent with the existing repository state. The review of these provided files is a mandatory prerequisite to verify the alignment of target framework monikers (TFMs), specifically verifying the net8.0-android or net9.0-android SDK targets, and to audit the current state of third-party NuGet packages such as sqlite-net-sqlcipher and Plugin.Maui.PeerVideoCall.[6]

The methodology requires that all external updates, platform-specific API deprecations, and library version increments are evaluated against this provided source of truth. If the provided codebase contains legacy implementations of background services, they will be systematically refactored to comply with the latest Android 15 foreground service mandates. The collaborative development process remains cautious regarding versioning, explicitly requesting repository access or compressed project archives prior to the commencement of any refactoring phase.

Architectural Blueprint for the.NET MAUI Android Background Service

The core of the application relies on an always-on networking daemon capable of initiating automatically when the device boots. In the.NET MAUI ecosystem, this requires deep integration with the underlying Android application lifecycle.[3] The system utilizes a BroadcastReceiver written in C# and decorated with the and attributes.[8] The application manifest must correspondingly declare the android.permission.RECEIVE_BOOT_COMPLETED permission.[8] Upon intercepting the boot sequence, this receiver triggers the instantiation of the

background service.

Navigating Android 15 Foreground Service Restrictions

Historically, Android permitted background services to run silently, but aggressive power management algorithms and privacy models have fundamentally altered this paradigm.[10] For an application targeting Android 14 (API level 34) and Android 15 (API level 35), the service must be explicitly elevated to a Foreground Service (FGS), which mandates the display of a persistent notification to ensure user awareness.[11] Furthermore, Android 15 introduces severe restrictions on the ability to launch these services from a BOOT_COMPLETED receiver.[12]

If the service were erroneously categorized under the dataSync, mediaPlayback, or phoneCall foreground service types, the Android 15 operating system would instantly terminate the application, throwing a fatal ForegroundServiceStartNotAllowedException upon boot.[12] Additionally, types such as dataSync are now strictly monitored and subjected to a maximum runtime limit of six hours within any twenty-four-hour period; exceeding this limit results in a hard timeout and an Application Not Responding (ANR) crash.[14]

Even though the daemon acts as a generic listener for upcoming data payloads, categorizing it under the dataSync type is not viable because Android 15 explicitly bans dataSync services from launching via a BOOT_COMPLETED broadcast receiver [34]. To fulfill the requirement of an uninterrupted, auto-starting P2P communication service, the architecture dictates the use of the remoteMessaging foreground service type, which remains exempt from these boot limitations [34].

Foreground Service Type	**Primary OS Intended Use Case**	**Boot Exemption Status (Android 15)**	**Runtime Limitation**	**Source**
dataSync	Cloud data synchronization and local file backups.	**Banned** on BOOT_COMPLETED.	Strict 6-hour limit per 24 hours.	[12], [34]
connectedDevice	Maintaining connections to Bluetooth or peripheral hardware.	Allowed on BOOT_COMPLETED.	No strict timeout enforced.	[15]

shortService	Critical, non-interruptible asynchronous tasks.	Allowed on BOOT_COMPLETED.	Strict 3-minute limit.	[17]
remoteMessaging	Transferring text messages between peer devices.	**Allowed** on BOOT_COMPLETED.	No strict timeout enforced.	[17], [34]

By declaring android:foregroundServiceType="remoteMessaging" in the manifest and requesting the FOREGROUND_SERVICE_REMOTE_MESSAGING permission, the.NET MAUI service can legally initiate upon boot and maintain a persistent network socket.[17] The C# implementation of the service extends the native Android.App.Service class, overriding the OnStartCommand method to construct an Android.App.Notification and subsequently calling StartForeground().[3]

Feature 1: Cryptographic Identity and SIM-Bound Login

The foundational requirement specifies that a user is uniquely identified by the mobile telecommunications number physically present in their device. The login and signup mechanisms bypass traditional email-and-password architectures, utilizing the cellular network as the primary identity anchor. During the initial application launch, the user inputs their name and desired username, while the system programmatically reads the mobile number using the Android TelephonyManager API, subject to the READ_PHONE_STATE permission.

Unique Identifier Generation

To facilitate secure P2P routing, the "uniquely identified id" requested in the requirements must transcend a simple alphanumeric string. The architecture implements this unique ID as an Ed25519 public key. During the signup phase, the application generates a cryptographically secure pseudorandom number generator (CSPRNG) seed, deriving an asymmetric key pair. The private key remains heavily encrypted within the local device storage, while the public key serves as the public-facing uniqueId. This ensures that all subsequent communications routed to this ID can be mathematically verified and end-to-end encrypted.[18]

Regulatory Compliance and SIM-Binding Directives

When engineering mobile-number-based authentication, the architecture must account for stringent telecommunications regulations, particularly those implemented to combat

cyber-fraud. Directives issued by the Department of Telecommunications (DoT) in jurisdictions such as India (effective throughout 2026) mandate strict "SIM-binding" for app-based communication services.[19]

These directives require that the communication application remains continuously linked to the physical Subscriber Identity Module (SIM) card utilized during the initial registration.[19] The.NET MAUI application must therefore implement a background validation loop that interfaces with the SubscriptionManager. By securely hashing the Integrated Circuit Card Identifier (ICCID) of the installed SIM during signup, the application can periodically verify its presence. If the SIM card is removed, deactivated, or swapped, the application must immediately sever the P2P connections and lock the encrypted local database, explicitly complying with the mandate that makes it "impossible to use the app without that active SIM".[21] Furthermore, any web-based or secondary device sessions interfacing with this service must programmatically enforce an automatic logout every six hours, requiring re-authentication via the primary device.[20]

Feature 2: Secure Local Storage Architecture

The second requirement dictates that all application data must be persisted in local storage and rendered entirely inaccessible until the user successfully authenticates using their mobile credentials. Given the vulnerabilities associated with physical device extraction and rooted file system access, utilizing standard SQLite databases or unencrypted .xml preference files is a severe security risk.[22]

To achieve enterprise-grade data at rest encryption within the.NET MAUI environment, the architecture integrates SQLCipher.[6] SQLCipher provides a transparent, drop-in replacement for the standard SQLite library, enforcing 256-bit Advanced Encryption Standard (AES) encryption on all database pages before they are written to the physical flash memory.[6]

Cryptographic Key Management via Android Keystore

The efficacy of SQLCipher relies entirely on the security of its encryption key. Hardcoding this password or storing it in plaintext SharedPreferences invalidates the encryption entirely.[23] The architecture resolves this by generating a highly entropic 32-byte (256-bit) key at runtime.[23] This master database key is then stored utilizing the .NET MAUI SecureStorage API, which interfaces directly with the hardware-backed Android Keystore system via the EncryptedSharedPreferences wrapper.[2]

Storage Component	Cryptographic Mechanism	Purpose	Source

Relational Data	SQLCipher (256-bit AES)	Encrypts all data payloads, peer public keys, and usernames at rest.	[6]
Database Key	Android Keystore (TEE/SE backed)	Secures the AES key used by SQLCipher, preventing extraction.	[24]
Authentication Gate	BiometricPrompt API / PIN	Requires user verification before releasing the Keystore key to memory.	[25]

Upon application launch, the background service remains in a dormant state regarding data access. The user must provide authentication—either through an SMS One-Time Password (OTP) validation of their phone number or via a local biometric prompt (fingerprint/facial recognition)—to unlock the Keystore.[25] Only upon successful authentication does the Keystore release the 32-byte symmetric key, allowing the.NET MAUI Entity Framework Core instance to instantiate the SQLCipher connection and access the local data.

Entity-Relationship Diagram (ERD)

To conceptualize the encrypted local data structures, the following Entity-Relationship Diagram details the generic, data-agnostic SQLite schema stored on each individual user's device.

Code snippet

```
erDiagram
    LOCAL_ACCOUNT {
        string MobileNumber PK "Primary Identity"
        string Name
        string UserName
        string UniqueId "Ed25519 Public Key"
        blob EncryptedPrivateKey "Secured by Android Keystore"
        string SimIccidHash "For continuous SIM-binding validation"
```

```
  }

  PEER {
    string UniqueId PK "Ed25519 Public Key"
    string MobileNumber UK "Discovered via SMS Handshake"
    string Name
    boolean IsTrusted
  }

  DATA_PAYLOAD {
    string PayloadId PK
    string SenderId FK "References Peer.UniqueId"
    string TargetId "Peer UniqueId or GroupId"
    blob EncryptedData
    int HopCountTTL "Gossip Protocol TTL"
    string PayloadHash "Bloom filter deduplication hash"
    datetime Timestamp
  }

  GROUP_SESSION {
    string GroupId PK
    string GroupName
    int CurrentEpoch "Message Layer Security (MLS) Epoch"
    blob RootSecret "Current TreeKEM shared secret"
  }

  GROUP_MEMBER {
    string GroupId FK
    string PeerUniqueId FK
  }

  LOCAL_ACCOUNT |

|--o{ DATA_PAYLOAD : "authors"
  PEER |

|--o{ DATA_PAYLOAD : "relays/authors"
  GROUP_SESSION |

|--|{ GROUP_MEMBER : "contains"
  PEER |

|--o{ GROUP_MEMBER : "participates in"
```

Feature 3: User Search and Out-of-Band SMS Handshake

In a traditional centralized application, user search is facilitated by querying a server database. In a pure decentralized P2P network, discovering the IP address and public key of a peer using only their mobile number requires an out-of-band communication channel.[5] The requirement specifies that adding a user triggers an SMS transmission; the recipient accepts the SMS and responds with their name and unique ID, establishing the connection matrix.

The Cryptographic Challenge-Response Protocol

The architecture utilizes the Short Message Service (SMS) infrastructure to execute a serverless cryptographic handshake.[5] When User A initiates a search for User B's mobile number, the background service generates an ephemeral Diffie-Hellman public key or extracts the persistent Ed25519 public key.[26] This key, concatenated with User A's uniqueId and current network routing hints (such as a local IP address and STUN-derived public IP), is serialized, compressed, and encoded into a Base64 string.

User A's device utilizes the native Android SmsManager API to dispatch this payload as a silent, background SMS. To prevent the SMS from appearing as gibberish in the recipient's default messaging application, the payload is prefixed with a specific deep-link URI scheme (e.g., p2p-app://handshake?data=...). User B's background service utilizes a BroadcastReceiver listening for the android.provider.Telephony.SMS_RECEIVED intent to intercept this specific format. Upon interception, the application prompts User B to accept the connection. If accepted, User B's device generates a reciprocal SMS containing their name and uniqueId (public key). Once both devices exchange these unique IDs, a secure, authenticated P2P WebRTC data channel or raw TCP socket is established, and all subsequent communications bypass the cellular SMS network entirely, shifting to the IP layer.[7]

Navigating Android 15 Restricted Settings for SMS

The programmatic transmission and interception of SMS payloads rely on the SEND_SMS and RECEIVE_SMS permissions.[27] However, Android 15 has fundamentally altered the permission landscape, classifying these as "hard restricted permissions" for applications that are side-loaded or installed from outside the Google Play Store.[28]

If the application attempts to prompt the user for SMS permissions using the standard ActivityCompat.requestPermissions dialog, the "Allow" button will be visually disabled by the operating system, preventing the handshake protocol from functioning.[28] To ensure a seamless onboarding experience, the application must act as a UI/UX expert, programmatically guiding the user through the Android OS "Restricted Settings" bypass via a meticulously designed user

flow:

1. **Detection and Redirection:** The application checks the permission status. Upon detecting the restricted state, it presents an educational, full-screen overlay explaining the necessity of SMS for P2P routing. A button utilizing Settings.ACTION_APPLICATION_DETAILS_SETTINGS routes the user directly to the application's specific system settings page.[28]
2. **Unlocking Restrictions:** The overlay provides an animated graphic instructing the user to tap the three-dot menu (⋮) located in the top-right corner of the App Info page and select "Allow restricted settings".[28] The UX accommodates edge cases, noting that on certain manufacturer overlays (e.g., Motorola), this menu is nested within the "Permissions" sub-screen.[28]
3. **Final Authorization:** Following the unlocking of restricted settings, the user is instructed to navigate to the 'Permissions' menu, locate 'SMS', and explicitly toggle the setting to "Allow".[28]

Feature 4: Inter-Process Communication (IPC) and Data Routing

The fourth requirement establishes the background service as a centralized networking router for other, third-party user-facing applications installed on the same device. This necessitates a mechanism for these disparate applications to send and receive arbitrary data payloads through the daemon service. Because the Android operating system isolates every application within its own Dalvik/ART virtual machine sandbox, direct memory access is prohibited.[25]

Data-Agnostic Inter-Process Communication (AIDL)

To allow various third-party apps to communicate with the background service concurrently, the interface contract must be entirely data-agnostic. The architecture dictates the use of the Android Interface Definition Language (AIDL).[31] AIDL decomposes objects into primitives that the operating system can understand and marshals them across process boundaries [31].

By designing the AIDL interface to accept generic data types (e.g., raw byte arrays), the daemon remains completely decoupled from the specific business logic of any client applications (such as chat apps). The API contract is designed using asynchronous patterns to prevent the client applications' main UI threads from blocking while waiting for network packet transmission.[33] Utilizing the oneway keyword in the AIDL definition ensures that calls from the client to the background service are non-blocking.

The architectural blueprint of the generic AIDL interface dictates the following structure:

Code snippet

```
package com.p2p.daemon.ipc;

import com.p2p.daemon.ipc.IDataReceiver;

interface IP2PNetworkService {
    // Authenticate the client application
    int registerClient(String appId, IDataReceiver receiver);

    // Transmit a 1-to-1 data payload asynchronously
    oneway void sendPayload(String targetUniqueId, in byte payload);

    // Broadcast a data payload to a group matrix
    oneway void sendGroupPayload(in List<String> targetIds, in byte payload);
}
```

When a third-party application binds to the service via Context.BindService, it provides a callback interface (IDataReceiver) allowing the background daemon to asynchronously push any incoming P2P network data directly to the client app [31].

Decentralized Data Propagation and Group Routing

Routing payloads in a decentralized topology requires sophisticated algorithms to prevent network congestion and guarantee delivery without a central server. The service implements an Epidemic Broadcast (Gossip) algorithm for data propagation.[18]

When a client application pushes a group data payload to the daemon via AIDL, the daemon utilizes a configurable fanout mechanism. The node signs the payload cryptographically using its Ed25519 private key and forwards it to N randomly selected, actively connected peers.[18] Those peers subsequently forward the payload to their subset of peers. This exponential propagation ensures that the payload reaches the entire network in $O(\log N)$ hops.[18]

To mitigate the risk of broadcast storms, the protocol enforces three strict parameters:

1. **Configurable Time-To-Live (TTL):** Every payload originates with a predefined hop counter. Each relay node decrements this counter, dropping the packet when the TTL reaches zero.[18]
2. **Bloom Filter Deduplication:** Every peer maintains a localized Bloom filter containing the cryptographic hashes of recently processed payloads. If a received payload hash matches the filter, it is silently discarded as a duplicate.[18]

3. **Signature Verification:** To prevent malicious actors from spoofing data, every relayed payload must possess a valid Ed25519 signature matching the sender's public uniqueId.[18]

Message Layer Security (MLS) for Group Scalability

To optimize the cryptography of generalized group payloads within the P2P architecture, the implementation of the Internet Engineering Task Force (IETF) Message Layer Security (MLS) standard (RFC 9420) is integrated into the routing logic.[26] MLS utilizes a protocol known as TreeKEM, organizing group members into a binary tree structure where leaf nodes represent individual participants and parent nodes represent shared symmetric secrets.[26]

When a user transmits a group payload, they encrypt the data only once, utilizing the root secret derived from the current cryptographic "epoch".[26] All authorized group members possess the necessary key derivation paths to independently calculate this root secret, allowing them to decrypt the payload locally.[26]

Development and Debugging Configuration

The explicit requirement to initially create this application for development and debugging necessitates the inclusion of specialized diagnostic modalities within the codebase. Because decentralized networks are notoriously difficult to trace due to the absence of centralized server logs, the background service implements extensive local telemetry.

The.NET MAUI dependency injection container is configured to inject comprehensive logging services that output directly to the Android Logcat stream, filtering by verbose custom tags identifying distinct protocol phases. To facilitate rapid iteration during the debugging phase, the strict SIM-binding and 6-hour timeout validation loops required for DoT compliance are isolated behind compilation directives (#if DEBUG) and can be bypassed.[19] Furthermore, the SMS handshake protocol is abstracted behind an interface. In the debugging configuration, the application provides a MockSmsManager that intercepts the Base64 payloads and loops them back via local intent broadcasts, allowing developers to simulate the exchange of uniqueId public keys between virtual emulator instances without incurring actual cellular SMS charges.

Works cited

1. GitHub - jsuarezruiz/awesome-dotnet-maui: A curated list of awesome .NET MAUI libraries and resources., accessed on March 7, 2026, https://github.com/jsuarezruiz/awesome-dotnet-maui
2. Connectivity - .NET MAUI - Microsoft Learn, accessed on March 7, 2026, https://learn.microsoft.com/en-us/dotnet/maui/platform-integration/communication/networking?view=net-maui-10.0
3. How to create a background service in .NET Maui - Stack Overflow, accessed on March 7, 2026, https://stackoverflow.com/questions/71259615/how-to-create-a-background-ser

vice-in-net-maui
4. WifiP2pManager Class (Android.Net.Wifi.P2p) | Microsoft Learn, accessed on March 7, 2026, https://learn.microsoft.com/en-us/dotnet/api/android.net.wifi.p2p.wifip2pmanager?view=net-android-35.0
5. P2P hand-shaking without a central server - Stack Overflow, accessed on March 7, 2026, https://stackoverflow.com/questions/23810734/p2p-hand-shaking-without-a-central-server
6. Ciphering Your Room: Quick Guide To Encrypting Data In Android Room Database Using SQLCipher | by Jesseosile | System Weakness, accessed on March 7, 2026, https://systemweakness.com/ciphering-your-room-quick-guide-to-encrypting-data-in-android-room-database-using-sqlcipher-618b80090a97
7. WebRTC .NET Maui Peer to Peer Video Calls! With just 3-4 lines of code! - C# Corner, accessed on March 7, 2026, https://www.c-sharpcorner.com/article/webrtc-net-maui-peer-to-peer-video-calls-with-just-3-4-lines-of-code/
8. NET MAUI AndroidManifest entry for RECEIVE_BOOT_COMPLETED Receiver, accessed on March 7, 2026, https://stackoverflow.com/questions/79778162/net-maui-androidmanifest-entry-for-receive-boot-completed-receiver
9. How implement to can auto start the application when i configure in settings of Android? · dotnet maui · Discussion #14960 - GitHub, accessed on March 7, 2026, https://github.com/dotnet/maui/discussions/14960
10. Android BLE Scanning in 2026: Why Your App Stops Finding Devices in the Background (And How to Fix It) - Medium, accessed on March 7, 2026, https://medium.com/@bleadvertiserapp/android-ble-scanning-in-2026-why-your-app-stops-finding-devices-in-the-background-and-how-to-fix-ba5ae06c17c3
11. Services overview | Background work - Android Developers, accessed on March 7, 2026, https://developer.android.com/develop/background-work/services
12. Behavior changes: Apps targeting Android 15 or higher | Android ..., accessed on March 7, 2026, https://developer.android.com/about/versions/15/behavior-changes-15#fgs-boot-completed-restrictions
13. Behavior changes: Apps targeting Android 15 or higher | Android ..., accessed on March 7, 2026, https://developer.android.com/about/versions/15/behavior-changes-15
14. Changes to foreground service types for Android 15, accessed on March 7, 2026, https://developer.android.com/about/versions/15/changes/foreground-service-types
15. Android Foreground Services: Types, Permissions and Limitations - Softices, accessed on March 7, 2026, https://softices.com/blogs/android-foreground-services-types-permissions-use-cases-limitations

16. Communicate in the background | Connectivity - Android Developers, accessed on March 7, 2026, https://developer.android.com/develop/connectivity/bluetooth/ble/background
17. Foreground service types | Background work - Android Developers, accessed on March 7, 2026, https://developer.android.com/develop/background-work/services/fgs/service-types
18. dicethedev/p2p-chat: A secure and decentralized peer-to-peer chat application with real-time messaging - GitHub, accessed on March 7, 2026, https://github.com/dicethedev/p2p-chat
19. Government tells WhatsApp, Telegram and others to link services to SIM cards: What it means for users - The Times of India, accessed on March 7, 2026, https://timesofindia.indiatimes.com/technology/tech-news/government-tells-whatsapp-telegram-and-others-to-link-services-to-sim-cards-what-it-means-for-users/articleshow/125673507.cms
20. India mandates SIM-linked messaging apps to fight rising fraud - Security Affairs, accessed on March 7, 2026, https://securityaffairs.com/185265/laws-and-regulations/india-mandates-sim-linked-messaging-apps-to-fight-rising-fraud.html
21. India Orders Messaging Apps to Work Only With Active SIM Cards to Prevent Fraud and Misuse - The Hacker News, accessed on March 7, 2026, https://thehackernews.com/2025/12/india-orders-messaging-apps-to-work.html
22. Android: Room: no encryption and security? - Stack Overflow, accessed on March 7, 2026, https://stackoverflow.com/questions/48799792/android-room-no-encryption-and-security
23. How to Encrypt Your Room Database in Android Using SQLCipher | by Pouya Heydari, accessed on March 7, 2026, https://proandroiddev.com/how-to-encrypt-your-room-database-in-android-using-sqlcipher-0bce78328bd6
24. How to Secure Android App Local Storage Against Hidden Risks - Appknox, accessed on March 7, 2026, https://www.appknox.com/blog/insecure-local-storage-android
25. Security checklist - Android Developers, accessed on March 7, 2026, https://developer.android.com/privacy-and-security/security-tips
26. Adapting the Signal Protocol for P2P Messaging | positive-intentions, accessed on March 7, 2026, https://positive-intentions.com/blog/p2p-signal-protocol/
27. Sending a Text Message Over the Phone Using SmsManager in Android - GeeksforGeeks, accessed on March 7, 2026, https://www.geeksforgeeks.org/android/sending-a-text-message-over-the-phone-using-smsmanager-in-android/
28. Android 15+ SEND_SMS Permission: How to Enable SMS Permissions on Android 15+, accessed on March 7, 2026, https://textbee.dev/blog/android-15-send-sms-permission-guide
29. How to grant SEND_SMS and RECEIVE_SMS permissions on Android 14+ -

httpSMS, accessed on March 7, 2026, https://httpsms.com/blog/grant-send-and-read-sms-permissions-on-android/
30. "SEND_SMS" Permission Not Allowed in App - Android 15 - Stack Overflow, accessed on March 7, 2026, https://stackoverflow.com/questions/79310952/send-sms-permission-not-allowed-in-app-android-15
31. Android Interface Definition Language (AIDL) | Background work - Android Developers, accessed on March 7, 2026, https://developer.android.com/develop/background-work/services/aidl
32. Mastering Inter-Process Communication (IPC) in Android - droidcon.com, accessed on March 7, 2026, https://www.droidcon.com/2025/07/07/mastering-inter-process-communication-ipc-in-android/
33. Using Kotlin APIs for Android Services with AIDLs - inovex GmbH, accessed on March 7, 2026, https://www.inovex.de/de/blog/concurrent-kotlin-apis-in-android-services/